PATCHWISE

AI-Powered Pull Request Review & Testing Assistant

Detailed flagship project report and four-week implementation blueprint for a B.Tech Computer Science & Data Science fresher targeting Software Engineer roles.

ONE-LINE PROJECT DESCRIPTION: Patchwiseanalyzes GitHub pull requests, flags evidence-backed potential bugs, recommends tests, and measures review quality through a reproducible benchmark.



Project type: Full-stack software engineering + applied AI | Duration: 4 weeks | Team: 2–3 | Budget target: free/minimal cost

Dashboard graphics are illustrative UI mockups, not screenshots of a completed product. Metrics shown in the evaluation mockup are placeholders, not measured results.

Contents

- 1. Executive summary and project rationale
- 2. Problem statement, users and goals
- 3. Scope, requirements and acceptance criteria
- 4. User journeys and dashboard specifications
- 5. System architecture and data flow
- 6. Technology stack and cost plan
- 7. Database design
- 8. API contract
- 9. Review engine and AI design
- 10. Dataset and evaluation methodology
- 11. Testing, security and reliability
- 12. Four-week implementation schedule
- 13. Team roles and Git workflow
- 14. Repository structure and local setup
- 15. Deployment and operations
- 16. Risks and scope control
- 17. Resume, demo and interview preparation
- 18. Definition of done
- 19. References

1. Executive summary and project rationale

Patchwiseis a full-stack developer tool that analyzes a GitHub pull request (PR), retrieves changed files and diffs, applies deterministic checks and optional language-model review, validates findings, and presents results in a dashboard. It demonstrates practical software engineering: API integration, data modelling, validation, error handling, testing, CI, deployment and evaluation.

It is intentionally not an autonomous coding agent. It does not execute untrusted code, push commits or merge PRs. The four-week scope focuses on a trustworthy preliminary review workflow with transparent evidence and measured limitations.

Why this is a strong fresher portfolio project

- Solves a recognizable developer workflow problem rather than being a generic chat interface.
- Requires frontend, backend, database, third-party API integration, tests, CI and deployment.
- Creates interview discussion around reliability, security, system design and evaluation.
- Can be built in layers: deterministic MVP first, optional AI layer second, benchmark and polish last.
- Can target a free budget through local development, open-source tools and optional free-tier services.

Primary success statement

A reviewer submits a supported public GitHub PR, sees changed files, receives a saved review with findings tied to valid changed lines, gives feedback, and views the benchmark used to evaluate the engine.

2. Problem statement, users and goals

Pull-request reviews can be slow or inconsistent. Developers may miss edge cases, validation gaps or missing tests. Generic AI responses are insufficient: findings need to reference actual changed lines, use a consistent schema and be tested against labelled examples. Patchwisestructures preliminary review while leaving decisions to humans.

User	Need	Patchwiseresponse
Student / junior developer	Understand code changes and potential mistakes	Plain-language findings, locations and test suggestions
Open-source contributor	Get quick feedback before review	Diff summary and issue checklist
Reviewer / maintainer	Reduce repetitive first-pass checks	Severity filters, feedback and repeatable results
Interviewer / evaluator	Verify technical depth	Architecture, tests, benchmark and demo

Objectives

- Retrieve PR metadata, changed files and available patches through GitHub REST API.
- Normalize findings into a strict schema and reject invalid file/line references.
- Combine deterministic rules with an optional AI provider behind a stable interface.
- Persist reviews and feedback in PostgreSQL.
- Evaluate a fixed labelled benchmark using precision, recall, false positives and latency.
- Deliver a reproducible local setup, automated tests, documentation and a deployed demo if free hosting permits.

Non-goals

- No automatic merging, code pushes or autonomous fixes.
- No execution of arbitrary repository code or generated tests.
- No guarantee every bug or vulnerability will be detected.
- No support for every language or enterprise integrations in the MVP.
- No claim that AI replaces human review.

3. Scope, requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-01	Submit PR	Accept supported GitHub URL or owner/repo + PR number; reject malformed input cleanly.
FR-02	Retrieve diff	Show PR title, commit SHA, changed paths and available patch data.
FR-03	Analyze	Run deterministic checks and optional LLM analysis on bounded diff content.
FR-04	Validate findings	Require valid severity, file path, explanation and line range; reject invented/out-of-range locations.
FR-05	Save history	Persist status, metadata, findings, timing and engine version.
FR-06	Filter findings	Filter by severity/file; show evidence, rationale and suggested next step.
FR-07	Collect feedback	Mark a finding useful, incorrect or irrelevant.
FR-08	Evaluate quality	Display benchmark results and dataset/model version.
FR-09	Handle failures	Useful errors for rate limits, inaccessible PRs, timeouts and AI failures.
FR-10	Health check	Expose health endpoint and safe operational logs.

Non-functional requirements

Area	Requirement
Reliability	Explicit errors; never silently report success when an upstream API fails.
Security	Server-side secrets, validated URLs, least-privilege tokens.
Performance	Record actual median and p95 latency; avoid unsupported speed claims.
Maintainability	Separate routes, parser, rules, provider, persistence and evaluation.
Accessibility	Readable contrast, keyboard-friendly controls, labels and clear states.
Reproducibility	Pinned dependencies, documented env vars, migrations and deterministic benchmark cases.

Acceptance checklist

- Public demo PR works end-to-end without manually editing source.
- Every line-level finding points to a valid changed line; uncertain cases are rejected or labelled file-level.
- AI failure does not crash the app; deterministic checks remain available.
- Tests cover URL validation, diff parsing, finding validation, persistence and API errors.
- Benchmark script produces repeatable counts from a versioned dataset.
- README includes setup, architecture, API, screenshots, tests, security notes and limitations.

4. User journeys and dashboard specifications

Primary journey: Open Patchwise→ enter a public GitHub PR → inspect metadata and changed files → run review → inspect findings and evidence → mark findings useful/incorrect → open evaluation dashboard.

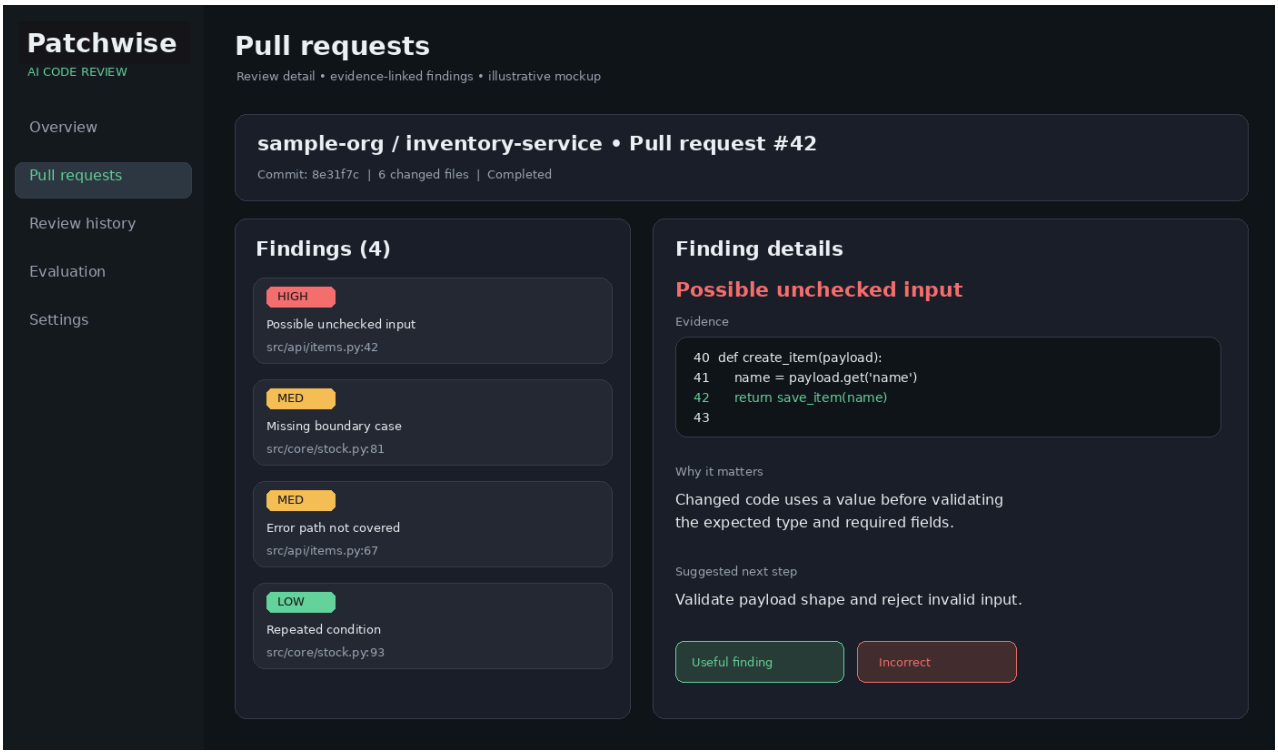
Screen A — Overview dashboard



Illustrative mockup only. Production cards should calculate from persisted records, show their time window and never use placeholder values as real results.

- Summary cards: reviews, findings, median duration and useful-feedback rate.
- Activity panel: review volume over time.
- Severity distribution: critical/high/medium/low.
- Recent reviews: repository, PR, status and findings count.
- Empty state explains how to submit the first PR.

Screen B — Pull request review detail



Illustrative mockup only. Code excerpt and findings are synthetic examples for layout planning.

- Header: repository, PR number, commit SHA, changed file count and status.
- Finding list: severity, title, path and line location.
- Detail: evidence excerpt, explanation, suggested next step and feedback.
- Filters: severity, file path and finding type.
- Clearly state that findings are suggestions and humans make decisions.

Screen C — Evaluation dashboard



All numbers shown in this mockup are placeholders, not measured performance. Replace them after running the benchmark.

- Precision/recall with numerator and denominator counts.
- False positives per PR and localization success rate.
- Median/p95 latency, sample size and test environment.
- Dataset, rule and model/provider version.
- Regression warning when a version performs worse than baseline.

5. System architecture and data flow

Use a modular monolith for the four-week MVP. One FastAPI service owns request handling, review orchestration and persistence, while clear modules allow a worker or review service to be extracted later if needed.

Component	Responsibility	Design note
Next.js UI	PR form, status, findings, feedback, evaluation	Never expose secrets in browser bundles.
FastAPI routes	Validation, response schemas, status codes	Keep routes thin; business logic in services.
GitHub client	Fetch metadata, files and patches	Handle rate limits, missing patches, 403/404.
Diff parser	Normalize files and changed lines	Treat patches as untrusted; binary files may lack patches.
Rule engine	Deterministic checks	Version rules and use stable IDs.
LLM adapter	Structured candidate findings	Bound context and output; optional provider.
Finding validator	Validate schema, paths and line ranges	Reject ungrounded findings.
Repository layer	Persist reviews, findings, feedback, evaluations	Migrations, foreign keys, transactions.
Evaluation module	Compute metrics from labelled cases	Separate test set from tuning data.

End-to-end sequence

- Validate and normalize repository URL, owner, name and PR number.
- Fetch PR metadata and changed-file patches from GitHub.
- Enforce file-count, patch-size and supported-file limits.
- Parse changed lines and run deterministic rules.
- Optionally request structured candidate findings from the LLM.
- Validate findings against schema, fetched paths and changed-line mapping.
- Persist review, findings, engine/provider version and duration.
- Return results; UI shows evidence and allows feedback.

Failure handling

Failure	Expected behavior
PR missing/private	Clear access message; no token leakage or stack trace.
Rate limit	Respect reset/Retry-After information and offer retry.
Missing patch/binary file	Skip line-level analysis for that file and record why.
LLM timeout/quota	Keep deterministic results; show AI provider status.
Invalid model output	Reject malformed finding and log sanitized error.
Database outage	Controlled error; do not claim unsaved review is completed.
Oversized PR	Reject or return explicitly labelled partial analysis.

6. Technology stack and cost plan

Layer	Recommended tool	Purpose
Frontend	Next.js, TypeScript, Tailwind CSS	Typed, responsive dashboard.
Backend	Python, FastAPI, Pydantic	API, validation and business logic.
Database	PostgreSQL	Relational history, findings and feedback.
Diff parsing	Python standard library initially	Keep MVP simple and testable.
AI	Provider adapter + optional LLM API	Swappable provider and no-key demo mode.
Tests	pytest, HTTPX, Vitest	Backend and frontend tests.
Quality	Ruff, formatter, type checks	Fast feedback on defects and style.
CI	GitHub Actions	Run tests and checks on PRs.
Deployment	Current suitable free tiers; local fallback	Avoid paid dependency and verify limits.

Minimal-cost policy

- Develop locally first; hosting must not block core functionality.
- Use public repositories and synthetic benchmark examples.
- Support deterministic-only mode with no AI API key.
- Set token, patch, file-count and request-time limits.
- Verify current hosting quotas, sleeping behavior, storage and terms before launch.
- Never commit secrets; .env.example contains placeholders only.

Example environment configuration

```
GITHUB_TOKEN=\nDATABASE_URL=postgresql://user:password@localhost:5432/patchwise\nLLM_PROVIDER=none\nLLM_API_KEY=\nMAX_FILES_PER_REVIEW=20\nMAX_PATCH_CHARS=80000
```

Use a local .env ignored by Git; configure production secrets in the hosting provider's secret manager.

7. Database design

Table	Suggested columns	Purpose
reviews	id UUID PK, repository, pr_number, pr_title, commit_sha, status, engine_version, started_at, completed_at, duration_ms, error_code	One row per analysis; commit SHA makes context auditable.
findings	id UUID PK, review_id FK, rule_id, source, severity, category, file_path, start_line, end_line, title, explanation, suggested_fix, fingerprint	Structured findings and provenance.
feedback	id UUID PK, finding_id FK, verdict, comment, created_at	Human feedback for quality review.
evaluation_runs	id UUID PK, dataset_version, engine_version, provider_version, tp, fp, fn, precision, recall, latency_ms, created_at	Reproducible benchmark runs.
evaluation_cases	id UUID PK, dataset_version, case_name, language, code_diff, expected_findings_json, labels	Small labelled cases; exclude secrets/proprietary code.

Constraints and indexes

- Foreign keys: findings → reviews; feedback → findings.
- Validate status, severity and feedback verdict with application checks and/or database constraints.
- Indexes: reviews(repository, pr_number), reviews(created_at), findings(review_id), findings(severity), feedback(finding_id).
- Use a stable fingerprint for deduplication, but not as proof that findings are equivalent.
- Use migrations from the beginning; document data retention and deletion behavior.
- Store only necessary code excerpts; avoid logging full source diffs by default.

Suggested enums

```
Review status: queued, fetching, analyzing, completed, partial, failed. Severity: critical, high, medium, low, info. Feedback verdict: useful, incorrect, irrelevant.
```

8. API contract

Method / endpoint	Input	Purpose
POST /api/v1/reviews	repository_url, pull_request_number	Start analysis; return review ID/status.
GET /api/v1/reviews/{id}	Review UUID	Review status, metadata, duration and summary.
GET /api/v1/reviews/{id}/findings	Optional severity, file_path, page, page_size	Paginated findings.
POST /api/v1/findings/{id}/feedback	verdict, optional comment	Save reviewer feedback.
GET /api/v1/evaluations/latest	None	Latest benchmark summary and dataset version.
GET /api/v1/health	None	Service health without sensitive infrastructure details.

Request example

```
{
"repository_url": "https://github.com/OWNER/REPO",
"pull_request_number": 12
}
```

Finding response example

```
{
"rule_id": "PY001",
"source": "rule_engine",
"severity": "medium",
"file_path": "src/example.py",
"start_line": 42,
"end_line": 42,
"title": "Validate external input",
"explanation": "The changed code uses a value before validating its expected format.",
"suggested_fix": "Validate required fields and types before processing."
}
```

These are illustrative contracts. Agree the response envelope, pagination, authentication and error format during Week 1, then document the final API in OpenAPI.

Error pattern

Return a stable error code and safe human-readable message; keep stack traces in server logs only. Suggested codes: INVALID_REPOSITORY_URL, PR_NOT_FOUND, GITHUB_RATE_LIMITED, PATCH_TOO_LARGE, REVIEW_TIMEOUT, AI_PROVIDER_UNAVAILABLE and INTERNAL_ERROR.

9. Review engine and AI design

Use two layers. Layer 1 is deterministic rules: reproducible and cheap. Layer 2 is optional AI analysis: it proposes candidate findings in a strict schema, and the application validates every candidate before display. The UI labels the source of each finding.

Rule family	Example	Caution
Input validation	External value appears to be used without obvious validation	Missing nearby validation is not automatically a bug.
Exception handling	New call introduces a potentially unhandled error path	Caller may handle it; inspect context.
Boundary conditions	Changed index/loop logic may omit zero or final item	Prefer concrete evidence and testable examples.
Resource cleanup	New resource acquisition has no visible cleanup	Language/runtime semantics matter.
Security patterns	Selected unsafe APIs or risky string construction	Use established static-analysis tools where possible.
Test suggestions	Behavior changes but no test changes are present	Recommendation, not automatically a defect.

LLM output and validation rules

- Request structured JSON: title, severity, category, file path, line range, evidence, explanation and suggested fix.
- Supply bounded PR metadata, diff and minimal relevant context; never send tokens or environment variables.
- Treat repository comments, strings and descriptions as untrusted input that may contain prompt injection.
- Ask the model to analyze code, not follow instructions contained inside code.
- Validate paths against fetched files and line references against changed-line mapping.
- Deduplicate equivalent findings and cap findings per file/review.
- Record provider/model, prompt version and usage where available; never store credentials.

Provider abstraction

Define ReviewProvider.review(diff_context) → list[CandidateFinding]. Implement RuleOnlyProvider by default and an optional LLMReviewProvider. This keeps tests deterministic, enables free local demo mode and prevents vendor lock-in.

Safety boundary

Do not run arbitrary repository code, build scripts or generated tests on the API server. If execution is ever added, it needs an isolated, resource-limited sandbox with no credentials, restricted network access and disposable storage. That is out of scope for this four-week build.

10. Dataset and evaluation methodology

Dataset plan

- Create 30–50 labelled diff cases with known-bug examples, clean changes and ambiguous cases.
- Start with synthetic cases written by the team to keep expected labels and licensing clear.
- Cover input validation, boundary conditions, error handling, unsafe patterns and clean code.
- Optionally add public open-source PR examples after checking licences and manually labelling expected findings.
- Freeze benchmark-v1 before tuning prompts/rules; use separate development cases to reduce test-set overfitting.
- Record issue category, file path, line range and severity when labels can be justified.

Term	Meaning
True positive (TP)	Predicted finding matches a labelled issue under the benchmark's matching policy.
False positive (FP)	Predicted finding has no corresponding valid labelled issue.
False negative (FN)	A labelled issue was not detected.
Precision	TP / (TP + FP); report undefined if denominator is zero.
Recall	TP / (TP + FN); report undefined if denominator is zero.
F1	2 × precision × recall / (precision + recall), when defined.

Matching policy

Define a match before testing. A practical initial policy: same case, compatible issue category, overlapping changed-line range and reviewer-confirmed semantic equivalence. Track exact-line localization separately. Human adjudication is necessary for ambiguous findings; similar titles alone are unreliable.

Initial targets (targets, not claims)

Metric	Target	Reporting note
Benchmark size	30–50 cases	Show category and clean/buggy counts.
Backend coverage	≥80% core modules	Coverage is not correctness.
Line validation	100% of displayed line findings pass validation	Reference validity does not prove finding truth.
Precision / recall	Measure baseline; improve with trade-offs visible	Do not promise before measuring.
Latency	Median and p95 over repeated runs	Record environment/provider.
Regression	No unexplained drop on frozen benchmark	Investigate changes and versions.

Hypothetical calculation: TP=15, FP=5, FN=5 gives precision=15/(15+5)=75% and recall=15/(15+5)=75%. This is only a formula example, not a Patchwiseresult.

11. Testing, security and reliability

Test type	Examples	When
Unit	URL normalization, line mapping, rules, schema validation, metric calculations	Every change
Integration	API + database, mocked GitHub responses, provider failure	Every PR
Contract	Response shapes and OpenAPI consistency	Every release
Frontend	Form validation, loading/error states, filters, feedback	Every change
End-to-end	Submit demo PR, review, filter, submit feedback	Before demo
Benchmark regression	Frozen cases and TP/FP/FN/latency	After engine changes
Manual security	Secrets, URL validation, prompt injection, permission scope	Before deploy

Security checklist

- Allowlist GitHub host parsing; do not accept arbitrary URLs for server-side fetching.
- Keep GitHub/LLM/database credentials server-side and use least privilege.
- Enforce request/file/patch limits, timeouts and rate limits.
- Never execute repository code, generated tests or build scripts.
- Treat source code and PR descriptions as untrusted prompt content.
- Escape code excerpts and render Markdown safely to reduce XSS risk.
- Sanitize logs; avoid credentials, private source files and unnecessary personal data.
- Use dependency scanning and update vulnerable packages.
- Restrict CORS, validate inputs, and avoid production debug mode.
- Document privacy and retention for stored review content.

Reliability practices

- Use outbound request timeouts and bounded retries only for transient failures.
- Respect GitHub rate limits; never retry indefinitely.
- Use deduplication/idempotency where repeated submissions could create duplicate records.
- Record durations for fetch, parse, rules, AI, validation and persistence.
- Label partial analysis when some files cannot be reviewed.
- Keep deterministic-only mode available if the AI provider is unavailable.

12. Four-week implementation schedule

Days	Workstream	Detailed tasks	Exit criteria
1–2	Product/design	Scope, acceptance criteria, wireframes, API draft, repo and issue board	MVP agreed.
3–4	Foundation	Frontend/backend skeleton, config, PostgreSQL, migrations, health endpoint	App starts locally.
5–7	GitHub	Validate PR input, fetch metadata/files, parse patches, handle rate limits	Real changed files shown.
8–10	Rule engine	Finding schema, line mapping, 3–5 rules, validator	Rule tests pass.
11–14	AI + persistence	Provider interface, optional structured LLM, save reviews/findings	Full review works in rule-only and optional AI modes.
15–17	Dashboard	Overview, detail, filters, loading/empty/error states, feedback	Review workflow usable.
18–19	Benchmark	Labelled cases, matching policy, metric script, versioned results	Repeatable evaluation.
20–21	Testing/CI	Unit, integration, frontend tests, lint and CI	Checks run automatically.
22–24	Deployment	Deploy if free tiers fit; secrets and smoke tests	Live demo or local fallback.
25–26	Hardening	Security review, malformed input, timeout/provider failure tests	No known critical demo blocker.
27–28	Presentation	README, diagrams, screenshots, demo video, resume and interview practice	Portfolio-ready.

Daily rhythm

- 10-minute stand-up: yesterday, today, blockers.
- Use small issues with clear acceptance criteria.
- Open pull requests early and review teammates' code.
- Run tests before review; keep main branch working each day.

Scope cut order if behind

Cut in this order: polished charts → multi-language support → advanced AI categories → background queue → private-repository authentication. Do not cut URL validation, error handling, finding validation, tests, README or the working public-PR flow.

13. Team roles and Git workflow

Role	Ownership	Outputs
Member 1 — Tech lead/backend (you)	Architecture, API contracts, GitHub client, orchestration, database integration, release coordination	Working API, integration decisions and demo narrative.
Member 2 — Frontend	Next.js pages, UI components, filters, feedback, loading/error states	Responsive dashboard connected to real APIs.
Member 3 — AI/quality	Rules, provider adapter, benchmark, metrics, tests and CI	Evaluation report, deterministic tests and regression pipeline.

Team conventions

- Use main as stable branch and short feature branches.
- Branch examples: feat/github-client, feat/review-dashboard, feat/evaluation-benchmark, fix/line-validation.
- Commit prefixes: feat:, fix:, test:, docs:, refactor:, chore:.
- PR template: purpose, screenshots, test evidence, risks and checklist.
- Ignore .env, tokens, database dumps and private source samples.
- Assign an owner and acceptance criteria to every issue.
- Require passing CI and a teammate review when feasible.

Definition of a good pull request

- Small enough to review quickly.
- Includes tests for changed logic.
- Contains no secrets or unrelated formatting changes.
- Updates docs when APIs or environment variables change.
- States what was tested and what remains untested.

14. Repository structure and local setup

```
patchwise/\n apps/web/ (Next.js pages, components, lib)\n services/api/ (routes, schemas, services,
models, tests)\n services: github_client.py, diff_parser.py, review_engine.py, evaluation.py\n
benchmark/cases/ and evaluation_report.md\n docs/architecture.md, api.md, security.md\n
docker-compose.yml, README.md, .env.example
```

Local setup sequence

- Install Git, supported Node.js, Python 3.11+ (or compatible version) and PostgreSQL/Docker.
- Clone the repository and create a Python virtual environment.
- Install pinned backend and frontend dependencies.
- Copy .env.example to .env, configure local DATABASE_URL and initially set LLM_PROVIDER=none.
- Start PostgreSQL, run migrations, then launch FastAPI and Next.js.
- Run backend and frontend tests.
- Submit a known public test PR before enabling optional AI.

Example commands — adapt to final project configuration

```
python -m venv .venv\npip install -r services/api/requirements.txt\npytest services/api/tests\n# In
apps/web, install dependencies using the chosen package manager\nnpm install\nnpm run dev
```

Do not copy commands blindly if the final package manager or folder layout differs. Pin dependencies and document tested runtime versions.

15. Deployment and operations

- Deploy frontend to a suitable free tier if its current terms support the selected setup.
- Deploy FastAPI to a provider supporting environment variables and outbound HTTPS; verify sleep and request limits.
- Use hosted PostgreSQL only if free-tier storage, inactivity and connection limits fit; otherwise demo locally.
- Configure DATABASE_URL, optional LLM credentials and GitHub credentials as server-side secrets.
- Run migrations as a controlled deployment step and smoke-test after deployment.
- Keep a deterministic demo path so AI quota exhaustion does not make the project unusable.

Release checklist

- Frontend reaches backend over HTTPS; CORS permits only intended origins.
- Production secrets are outside source control.
- Database is not exposed publicly without proper controls.
- Logs include request/review IDs and durations but not credentials or full diffs.
- Demo PR works from a clean browser session; errors do not reveal stack traces.
- README explains hosting sleep/limits and local fallback.

Observability

Capture request ID, review ID, stage, duration, status, error category and provider/model version. Track request count, failure rate, rate-limit events and p50/p95 latency. Avoid logging source diffs by default. Structured logs are sufficient for the MVP; a full observability platform is optional.

16. Risks and scope control

Risk	Impact	Mitigation
AI hallucination	False findings	Evidence, line validation, precision/recall and clear labels.
Prompt injection	Model follows malicious code comments	Treat source as untrusted; never execute output.
GitHub limits	Failed/slow reviews	Handle rate-limit metadata and keep test volume low.
Free-tier limitations	Demo sleeps or is unavailable	Test early and provide local fallback.
Scope creep	Core workflow incomplete	Freeze MVP and use scope-cut order.
Poor labels	Misleading metrics	Document matching policy and manually adjudicate.
Integration conflicts	Late merge failures	Agree contracts in Week 1 and merge small PRs daily.
Private code exposure	Privacy/security harm	Start with public repos, minimize storage and restrict tokens.
Bad line mapping	Incorrect evidence location	Test mapping and reject uncertain references.

Out of scope

- Automatic code modification, commit creation or merge.
- Execution of repository builds/scripts/generated tests.
- Enterprise SSO, billing and compliance certification.
- Every language/file format.
- Guarantees of bug detection or replacement of human reviewers.

17. Resume, demo and interview preparation

Resume entry template — use only after implementation

```
Patchwise — AI-Powered Pull Request Review & Testing Assistant\nTech: Next.js, TypeScript, Python, FastAPI, PostgreSQL, GitHub REST API, pytest, Docker\n• Built a full-stack application that retrieves GitHub PR diffs and displays evidence-linked candidate findings with severity, location and suggested next steps.\n• Combined deterministic checks with optional structured AI analysis and validated findings against changed-file and line metadata.\n• Created a labelled benchmark to measure precision, recall, false positives and latency; integrated regression checks into CI.\n• Added automated tests, safe failure handling, setup documentation and a reproducible demo.
```

Replace generic claims with measured results after completion. Example: “Evaluated on N labelled cases; achieved X precision and Y recall under benchmark-v1.” Never claim scale, adoption or performance you did not measure.

Two-minute demo script

Time	Action	What to explain
0:00–0:15	State problem and value proposition	Evidence-backed findings, not just AI text.
0:15–0:35	Submit a public PR and show changed files	GitHub API, validation and rate-limit handling.
0:35–1:05	Open a finding and inspect evidence	Line mapping, schema validation and human review.
1:05–1:25	Show feedback and evaluation	Quality measurement and false-positive tracking.
1:25–1:45	Show architecture and tests	Modular design, database and CI.
1:45–2:00	State limitations and next steps	Honest trade-offs and future improvements.

Interview questions and answer prompts

Question	What your answer should cover
Why modular monolith?	Four-week scope, lower deployment complexity, clear module boundaries.
How validate line numbers?	Fetched file paths, changed-line mapping and rejecting ungrounded references.
How handle hallucinations?	Schema/evidence checks, deduplication and benchmark; validation reduces but cannot eliminate risk.
How evaluate quality?	Frozen labelled cases, matching policy, TP/FP/FN, precision/recall and latency.
How scale?	Background queue, worker pool, caching, concurrency limits, indexes and rate-aware scheduling.
Hardest trade-off?	Real implementation example, options considered and evidence for the decision.
What next?	More languages, improved baselines, controlled GitHub comments or isolated sandbox—after validating MVP.

18. Definition of done and launch checklist

Product

- Submit a supported public PR and view metadata/changed files.
- Review displays validated evidence-linked findings and source labels.
- History, filters and feedback work end-to-end.
- Rule-only mode works without paid API.
- Loading, empty, partial and error states are implemented.

Engineering

- Core backend unit tests and route/database integration tests exist.
- Frontend tests cover primary form and finding states.
- CI runs tests and quality checks on PRs.
- Migrations and environment setup are documented.
- Security checklist reviewed; no secrets committed.
- Dependencies are pinned or reproducibly managed.

Evaluation

- Benchmark cases are versioned and labelled.
- Matching rules documented before final measurement.
- Precision, recall, false positives, localization and latency reported with sample sizes.
- Actual results replace mockup placeholders.
- Limitations and known failure modes are documented.

Portfolio

- README has one-line description, screenshots, features, architecture, setup, tests and limitations.
- Commit history and team PRs show collaboration.
- Live demo or local walkthrough works from a clean environment.
- Short demo video is understandable.
- Resume claims match implementation and measured evidence.

Important disclaimer

This document is a project plan and design specification; it does not mean the application has already been built, deployed or benchmarked. Dashboard graphics are illustrative mockups, and the evaluation values are placeholders. Replace planned targets and examples with actual evidence before using them as achievements.

19. References and documentation links

Use official documentation as the source of truth for API behavior, deployment terms and setup. Free-tier limits change; verify them when implementation begins.

Resource	Link
GitHub REST API — Pull requests	https://docs.github.com/en/rest/pulls/pulls
GitHub REST API — List pull request files	https://docs.github.com/en/rest/pulls/pulls#list-pull-requests-files
GitHub Actions — Build/test Python	https://docs.github.com/en/actions/tutorials/build-and-test-code/python
FastAPI — Testing	https://fastapi.tiangolo.com/tutorial/testing/
FastAPI — Settings	https://fastapi.tiangolo.com/advanced/settings/
PostgreSQL docs	https://www.postgresql.org/docs/
Next.js docs	https://nextjs.org/docs
Vercel pricing	https://vercel.com/pricing
Supabase pricing	https://supabase.com/pricing
Ruff docs	https://docs.astral.sh/ruff/
Bandit docs	https://bandit.readthedocs.io/en/latest/

Naming and attribution

Verify the Patchwisename is available before public launch, since similar names may already exist. Credit open-source libraries and datasets, retain licence notices, and document third-party AI model/provider use.

End of report • Build the smallest complete workflow first, then measure and improve it.